

UNIVERSIDADE FEDERAL DE
UBERLÂNDIA

PPGEELT – Pós-Graduação em Engenharia
Elétrica

LUCAS ALBINO MARTINS
12622EEL006

TRABALHO06 - APROXIMAÇÃO FUNCIONAL

Disciplina: EL056 - Redes Neurais Artificiais

Uberlândia

2026

**UNIVERSIDADE FEDERAL DE
UBERLÂNDIA**

PPGEELT – Pós-Graduação em Engenharia
Elétrica

LUCAS ALBINO MARTINS
12622EEL006

TRABALHO06 - APROXIMAÇÃO FUNCIONAL

Disciplina: EL056 - Redes Neurais Artificiais

Trabalho apresentado ao Programa de Pós-Graduação em Engenharia Elétrica (PPGEELT) da Universidade Federal de Uberlândia como requisito parcial de avaliação da disciplina.

Disciplina: EL056 - Redes Neurais Artificiais.

Professor: Prof. Dr. Keiji Yamanaka.

Uberlândia

2026

Aproximação funcional com Perceptron Multicamadas treinado por retropropagação do erro

1 Introdução

Aproximar uma função desconhecida $f : \mathbb{R} \rightarrow \mathbb{R}$ a partir de um conjunto finito de amostras $\{(x_p, t_p)\}_{p=1}^P$ é um dos problemas clássicos de redes neurais: a rede deve produzir uma função contínua que passe próxima dos pontos observados e que tenha comportamento razoável entre eles. Neste trabalho, os $P = 11$ pontos fornecidos no enunciado (Tabela 1) são aproximados por um Perceptron Multicamadas (MLP) treinado por retropropagação do erro (*backpropagation*), implementado integralmente em NumPy.

Tabela 1 – Pontos amostrados fornecidos no enunciado.

x	0	0,1	0,2	0,3	0,4	0,5	0,6	0,7	0,8	0,9	1,0
t	-0,9602	-0,5770	-0,0729	0,3771	0,6405	0,6600	0,4609	0,1336	-0,2013	-0,4344	-0,5000

Fonte: enunciado do Trabalho 06.

Além do treinamento da rede pedida, investigam-se o efeito do número de neurônios ocultos, da taxa de aprendizado, do termo de momento, da inicialização dos pesos e do modo de treinamento, bem como a capacidade de generalização e de extrapolação da rede obtida.

2 Fundamentação teórica

2.1 O Perceptron Multicamadas

Um neurônio artificial calcula a soma ponderada de suas entradas acrescida de um bias, $net = \mathbf{w}^\top \mathbf{x} + b$, e aplica uma função de ativação $\varphi(net)$ (HAYKIN, 2009). Um MLP organiza esses neurônios em camadas totalmente conectadas. Para o problema escalar deste trabalho usa-se a topologia 1– N –1: uma entrada, N neurônios ocultos com tangente hiperbólica e um neurônio de saída linear. Em notação matricial, para um lote $\mathbf{X} \in \mathbb{R}^{1 \times P}$:

$$\mathbf{net}^{(1)} = \mathbf{W}_1 \mathbf{X} + \mathbf{b}_1, \quad \mathbf{H} = \tanh(\mathbf{net}^{(1)}), \quad \mathbf{Y} = \mathbf{W}_2 \mathbf{H} + b_2, \quad (1)$$

com $\mathbf{W}_1, \mathbf{b}_1 \in \mathbb{R}^{N \times 1}$, $\mathbf{W}_2 \in \mathbb{R}^{1 \times N}$ e $b_2 \in \mathbb{R}$, totalizando $3N + 1$ parâmetros livres.

2.2 Funções de ativação

A tangente hiperbólica $\varphi(v) = \tanh(v)$ é sigmoideal, limitada em $(-1, 1)$, antissimétrica e diferenciável, com derivada expressa pela própria saída:

$$\varphi'(v) = 1 - \tanh^2(v) = 1 - h^2. \quad (2)$$

A antissimetria acelera o treinamento em relação à sigmoide logística (HAYKIN, 2009). Na saída adota-se a ativação linear $\varphi(v) = v$, com $\varphi'(v) = 1$. Assim a amplitude da resposta não fica limitada a $(-1, 1)$ e a rede implementa uma combinação linear de funções sigmoideais deslocadas e escaladas.

2.3 Teorema da Aproximação Universal

Cybenko (1989) demonstrou que somas finitas da forma

$$G(x) = \sum_{j=1}^N \alpha_j \sigma(w_j x + \theta_j) \quad (3)$$

com σ sigmoideal contínua são densas no espaço das funções contínuas sobre um compacto: para toda f contínua e todo $\varepsilon > 0$ existe N finito tal que $|G(x) - f(x)| < \varepsilon$ em todo o domínio. Hornik (1991) mostrou que a propriedade decorre da própria arquitetura em camadas, valendo para qualquer ativação limitada e não constante. A Eq. (3) é exatamente a rede 1- N -1 da Eq. (1).

O teorema é de *existência*. Ele não informa quantos neurônios são necessários, não garante que o gradiente descendente encontre os pesos adequados, não trata da generalização a partir de poucas amostras e não diz nada sobre o comportamento da rede fora do domínio compacto considerado. Esses aspectos são avaliados empiricamente na Seção 4.

2.4 Algoritmo de retropropagação

O treinamento minimiza o erro quadrático

$$E = \frac{1}{2P} \sum_{p=1}^P (t_p - y_p)^2, \quad (4)$$

cujos gradientes são obtidos pela regra da cadeia, propagando o erro da saída para a entrada (RUMELHART; HINTON; WILLIAMS, 1986). Definem-se os gradientes locais

$\delta = \partial E / \partial net$ (a menos do fator $1/P$). Na saída linear,

$$\boldsymbol{\delta}^{(2)} = \mathbf{Y} - \mathbf{T}, \quad (5)$$

e, na camada oculta, o erro é retropropagado pelos pesos $\mathbf{W}_2$ e modulado pela derivada da ativação:

$$\boldsymbol{\delta}^{(1)} = (\mathbf{W}_2^\top \boldsymbol{\delta}^{(2)}) \odot (1 - \mathbf{H}^2). \quad (6)$$

Os gradientes em relação aos parâmetros são o produto do δ de cada neurônio pelo sinal que chega a ele:

$$\frac{\partial E}{\partial \mathbf{W}_2} = \frac{1}{P} \boldsymbol{\delta}^{(2)} \mathbf{H}^\top, \quad \frac{\partial E}{\partial b_2} = \frac{1}{P} \sum_p \delta_p^{(2)}, \quad \frac{\partial E}{\partial \mathbf{W}_1} = \frac{1}{P} \boldsymbol{\delta}^{(1)} \mathbf{X}^\top, \quad \frac{\partial E}{\partial \mathbf{b}_1} = \frac{1}{P} \sum_p \delta_{:,p}^{(1)}. \quad (7)$$

A implementação foi validada comparando esses gradientes com diferenças finitas centradas. O erro relativo foi inferior a 10^{-8} em todos os parâmetros.

2.5 Gradiente descendente com momento

Os pesos são atualizados na direção oposta ao gradiente, com taxa de aprendizado α . O termo de momento β acumula uma fração da atualização anterior (RUMELHART; HINTON; WILLIAMS, 1986; HAYKIN, 2009):

$$\Delta \mathbf{w}(k) = -\alpha \nabla_{\mathbf{w}} E + \beta \Delta \mathbf{w}(k-1), \quad \mathbf{w}(k+1) = \mathbf{w}(k) + \Delta \mathbf{w}(k). \quad (8)$$

O momento funciona como um filtro passa-baixas sobre as atualizações: acelera o avanço em direções de gradiente consistente (platôs) e amortece oscilações em vales estreitos. No modo *em lote* faz-se uma atualização por época com o gradiente médio. No modo *online* (padrão a padrão) faz-se uma atualização por amostra, em ordem aleatória a cada época. O procedimento é resumido no Algoritmo 1.

Algoritmo 1 – Treinamento da MLP 1- N -1 por retropropagação com momento (modo em lote).

Input: amostras $(\mathbf{X}, \mathbf{T})$; N , α , β , tol , max_epocas

```

1 Inicializar  $\mathbf{W}_1, \mathbf{b}_1, \mathbf{W}_2, b_2$  (Nguyen-Widrow) e velocidades  $\Delta \mathbf{w} \leftarrow \mathbf{0}$ ;
2 for  $k \leftarrow 1$  to  $max\_epocas$  do
3    $\mathbf{H} \leftarrow \tanh(\mathbf{W}_1 \mathbf{X} + \mathbf{b}_1)$ ;  $\mathbf{Y} \leftarrow \mathbf{W}_2 \mathbf{H} + b_2$ ;                                // direta
4    $\boldsymbol{\delta}^{(2)} \leftarrow \mathbf{Y} - \mathbf{T}$ ;  $\boldsymbol{\delta}^{(1)} \leftarrow (\mathbf{W}_2^\top \boldsymbol{\delta}^{(2)}) \odot (1 - \mathbf{H}^2)$ ;                // retropropag.
5   foreach parâmetro  $\mathbf{w}$  com gradiente  $\mathbf{g}$  do
6      $\Delta \mathbf{w} \leftarrow \beta \Delta \mathbf{w} - \alpha \mathbf{g}$ ;  $\mathbf{w} \leftarrow \mathbf{w} + \Delta \mathbf{w}$ ;
7   if  $MSE = \frac{1}{P} \sum_p (t_p - y_p)^2 < tol$  then parar;
```

2.6 Inicialização de Nguyen–Widrow

Com pesos pequenos e aleatórios, as regiões de maior inclinação das $\tanh$ (onde $|net|$ é pequeno) tendem a se concentrar em uma mesma parte do domínio, e vários neurônios acabam fazendo o mesmo papel. Nguyen e Widrow (1990) propuseram redimensionar os pesos da camada oculta para que essas regiões ativas se distribuam ao longo do intervalo de entrada. Com n entradas e N neurônios ocultos, define-se $\beta_{NW} = 0,7 N^{1/n}$, sorteia-se $\mathbf{w}_j \sim \mathcal{U}(-0,5; 0,5)$, normaliza-se cada vetor para $\|\mathbf{w}_j\| = \beta_{NW}$ e sorteiam-se os bias em $\mathcal{U}(-\beta_{NW}; \beta_{NW})$ (FAUSETT, 1994).

2.7 Viés, variância e generalização

Uma rede com poucos neurônios tem viés alto e não consegue representar a forma da função (subajuste). Uma rede com muitos parâmetros em relação ao número de amostras tem variância alta e pode interpolar os pontos oscilando entre eles (sobreajuste). Com apenas 11 amostras, o erro de treino não serve para escolher N , porque ele sempre diminui com a capacidade da rede. Por isso a generalização foi estimada por validação cruzada *leave-one-out* (LOO): para cada ponto i , treina-se a rede com os outros 10 pontos e mede-se o erro quadrático em x_i . O MSE-LOO é a média desses 11 erros.

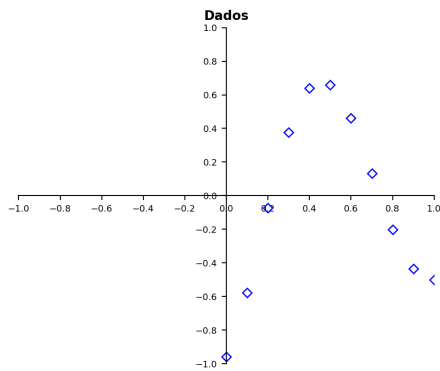
3 Metodologia

A rede, a retropropagação e o otimizador foram implementados do zero em Python/-NumPy. O programa está no arquivo `trabalho06_mlp.py` (Apêndice A), e a demonstração passo a passo, no *notebook* `trabalho06_mlp.ipynb`. A configuração base foi: MLP 1–5–1, $\tanh$ na camada oculta, saída linear, inicialização de Nguyen–Widrow, treinamento em lote, $\alpha = 0,05$, $\beta = 0,9$, parada em $\text{MSE} < 10^{-3}$ ou após 20 000 épocas, e semente 42. Os experimentos foram os seguintes: (i) treinamento base; (ii) $N \in \{1, 2, 3, 5, 10, 20\}$ treinados por 50 000 épocas, sem parada antecipada; (iii) $\alpha \in \{0,01; 0,05; 0,1; 0,3\}$ combinado com $\beta \in \{0; 0,5; 0,9\}$, medindo as épocas até $\text{MSE} < 10^{-3}$ (mediana de 10 sementes); (iv) inicialização uniforme $\times$ Nguyen–Widrow e modo em lote $\times$ online (10 sementes); (v) MSE-LOO para cada N ; e (vi) avaliação da rede em $x \in [-1, 1]$ para analisar a extrapolação.

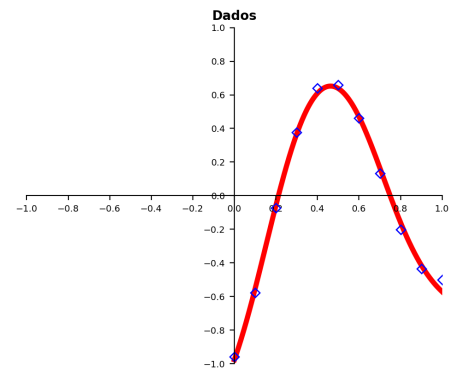
4 Resultados e discussão

4.1 Treinamento base

A rede 1–5–1 atingiu $\text{MSE} = 9,99 \times 10^{-4}$ em **89 épocas**. A Figura 1 reproduz o formato das figuras do enunciado. A curva aprendida é suave, tem pico próximo de $x \approx 0,46$ e passa próxima dos 11 pontos. O maior desvio ocorre em $x = 1,0$ ($y = -0,572$ para $t = -0,500$). Esse ajuste é mais fiel aos pontos do que a curva ilustrativa do enunciado, cujo pico fica perto de 0,5 e que se afasta visivelmente dos pontos em $x = 0$ e $x \in \{0,4; 0,5\}$. Continuando o treinamento sem parada antecipada, o MSE cai para $8,2 \times 10^{-5}$ em 50 000 épocas (Figura 2), mas quase toda a redução acontece nas primeiras centenas de épocas. Os pesos finais estão na Tabela 2.



(a) Pontos amostrados.



(b) Aproximação pela MLP 1–5–1.

Figura 1 – Dados do enunciado e função aproximada pela rede treinada.

Fonte: elaborado pelo autor.

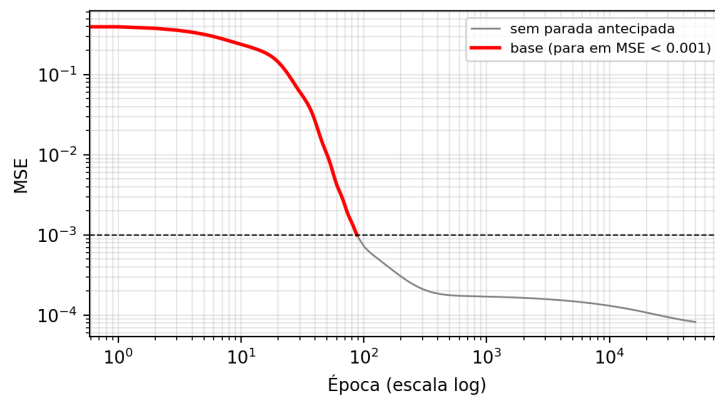


Figura 2 – Curva de aprendizado (MSE× época, escalas logarítmicas).

Fonte: elaborado pelo autor.

Tabela 2 – Pesos finais da rede base ($N = 5$, 89 épocas).

Neurônio oculto j	1	2	3	4	5
$w_j^{(1)}$	3,7694	-3,8397	3,4926	3,5000	-3,5034
$b_j^{(1)}$	-0,6305	2,6729	1,3174	2,2503	-0,3970
$w_j^{(2)}$	1,3552	0,9670	-0,7331	-0,2684	-0,2506
Bias da saída: $b_2 = -0,3750$					

Fonte: elaborado pelo autor.

4.2 Número de neurônios e generalização

A Figura 3 e a Tabela 3 mostram o efeito de N . Com $N = 1$ a rede só consegue representar um degrau sigmoidal (viés alto, MSE de 0,14). Dois neurônios já capturam o formato de sino, porque a diferença de duas sigmóides deslocadas forma uma “lombada”. O erro de treino cai de forma monótona com N , e com $N = 20$ (61 parâmetros para 11 pontos) a rede interpola os dados até a precisão de máquina. O MSE-LOO (Figura 4a), por outro lado, é mínimo em $N = 5$ ($8,4 \times 10^{-4}$) e piora 30 vezes em $N = 20$ ($2,7 \times 10^{-2}$). Nesse caso, a rede passa exatamente pelos pontos de treino, mas erra os pontos retirados. Esse é o dilema viés-variância e justifica a escolha de $N = 5$.

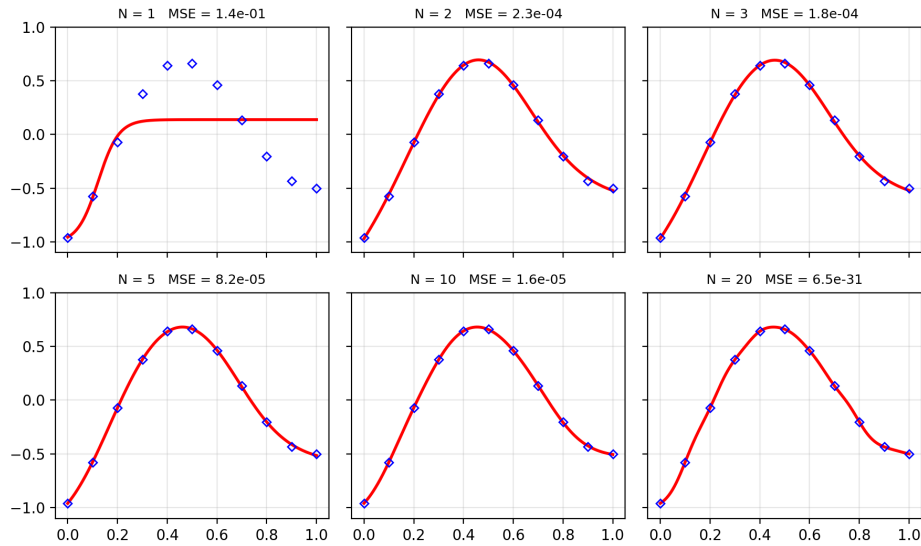


Figura 3 – Aproximação obtida para diferentes números de neurônios ocultos (50 000 épocas).

Fonte: elaborado pelo autor.

Tabela 3 – Erro de treino e erro de validação *leave-one-out* em função de N .

N	Parâmetros	MSE treino	MSE LOO	Maior $ e $ LOO
1	4	$1,41 \times 10^{-1}$	$3,31 \times 10^{-1}$	1,210
2	7	$2,27 \times 10^{-4}$	$2,61 \times 10^{-3}$	0,118
3	10	$1,81 \times 10^{-4}$	$2,86 \times 10^{-3}$	0,114
5	16	$8,22 \times 10^{-5}$	$8,43 \times 10^{-4}$	0,076
10	31	$1,56 \times 10^{-5}$	$1,31 \times 10^{-3}$	0,082
20	61	≈ 0 ($6,5 \times 10^{-31}$)	$2,68 \times 10^{-2}$	0,350

Fonte: elaborado pelo autor.

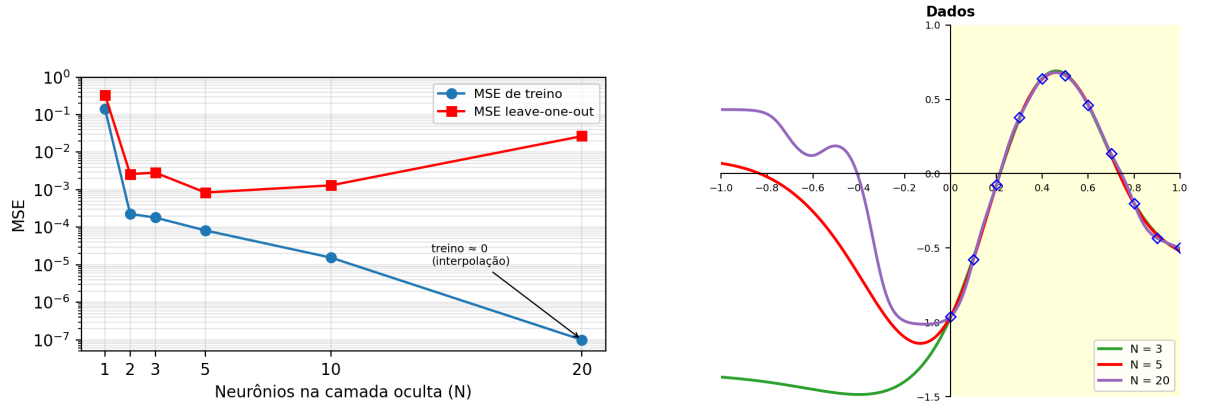
(a) Erro de treino e erro *leave-one-out* em função de N .(b) Redes treinadas avaliadas em $x \in [-1, 1]$.

Figura 4 – Generalização (validação cruzada) e extrapolação.

Fonte: elaborado pelo autor.

4.3 Taxa de aprendizado, momento, inicialização e modo de treinamento

A Tabela 4 mostra que o momento acelera a convergência em todas as taxas testadas. Com $\alpha = 0,01$, passar de $\beta = 0$ para $\beta = 0,9$ reduz a mediana de 6 732 para 747 épocas (cerca de 9 vezes). Isso é coerente com a interpretação do momento como um aumento da taxa efetiva, que se aproxima de $\alpha/(1 - \beta)$ em regiões de gradiente estável. Nenhuma combinação divergiu nesta faixa, porque o problema é pequeno e bem condicionado. As falhas registradas com $\beta \leq 0,5$ e $\alpha \leq 0,05$ correspondem a uma semente que ficou presa em um platô e não atingiu a tolerância em 50 000 épocas.

Tabela 4 – Épocas até $\text{MSE} < 10^{-3}$ (mediana de 10 sementes; entre parênteses, quantas convergiram).

α	$\beta = 0$	$\beta = 0,5$	$\beta = 0,9$
0,01	6 732 (9/10)	3 367 (9/10)	747 (10/10)
0,05	1 347 (9/10)	737 (10/10)	157 (10/10)
0,10	736 (10/10)	369 (10/10)	117 (10/10)
0,30	246 (10/10)	124 (10/10)	80 (10/10)

Fonte: elaborado pelo autor.

Tabela 5 – Inicialização e modo de treinamento (épocas até $\text{MSE} < 10^{-3}$, mediana de 10 sementes).

Inicialização	Em lote	Online
Uniforme	6 297	921
Nguyen–Widrow	157	25

Fonte: elaborado pelo autor.

A inicialização teve efeito ainda maior (Tabela 5). Com pesos uniformes em $(-0,5; 0,5)$ e entradas em $[0, 1]$, todas as tanh operam quase na região linear, e o treinamento em lote precisou de mediana de 6 297 épocas. Com Nguyen–Widrow foram 157 épocas, cerca de 40 vezes menos. No modo online a rede faz 11 atualizações por época, o que reduz o número de épocas. Comparando pelo número de atualizações de pesos, porém, a diferença é pequena com Nguyen–Widrow ($24,5 \times 11 \approx 270$ contra 157).

4.4 Extrapolação

A Figura 4b avalia redes com $N = 3, 5$ e 20 no intervalo $[-1, 1]$ dos eixos do enunciado. Dentro de $[0, 1]$ (faixa amarela) as três curvas praticamente coincidem. Fora desse intervalo elas divergem, e cada uma segue a saturação das suas próprias tanh. A rede com $N = 20$ ainda apresenta ondulações sem nenhuma relação com os dados. Esse comportamento ilustra o limite do Teorema da Aproximação Universal: a garantia vale apenas no domínio compacto coberto pelas amostras, e a rede não aprende a lei que gerou os dados.

5 Conclusão

Uma MLP 1–5–1 com tangente hiperbólica na camada oculta e saída linear, treinada por retropropagação com momento, aproximou os 11 pontos do enunciado com $\text{MSE} < 10^{-3}$ em 89 épocas. A curva obtida é suave e fiel aos dados, como pedido no trabalho. Os experimentos confirmaram a teoria: (i) o número de neurônios deve ser escolhido pelo erro de generalização e não pelo erro de treino, e a validação *leave-one-out* indicou $N = 5$ e mostrou sobreajuste com $N = 20$; (ii) o momento e a inicialização de Nguyen–Widrow reduzem o tempo de treinamento em uma e duas ordens de grandeza, respectivamente; e (iii) a aproximação só é confiável dentro do intervalo amostrado. Como extensões possíveis, ficam técnicas de regularização (*weight decay*, parada antecipada por validação) e otimizadores de segunda ordem, como Levenberg–Marquardt.

Referências

- CYBENKO, G. Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals and Systems*, v. 2, n. 4, p. 303–314, 1989.
- FAUSETT, L. *Fundamentals of neural networks: architectures, algorithms, and applications*. Englewood Cliffs: Prentice Hall, 1994.
- HAYKIN, S. *Neural networks and learning machines*. 3. ed. Upper Saddle River: Pearson, 2009.
- HORNIK, K. Approximation capabilities of multilayer feedforward networks. *Neural Networks*, v. 4, n. 2, p. 251–257, 1991.
- NGUYEN, D.; WIDROW, B. Improving the learning speed of 2-layer neural networks by choosing initial values of the adaptive weights. In: *Proceedings of the International Joint Conference on Neural Networks (IJCNN)*. San Diego: IEEE, 1990. v. 3, p. 21–26.
- RUMELHART, D. E.; HINTON, G. E.; WILLIAMS, R. J. Learning representations by back-propagating errors. *Nature*, v. 323, p. 533–536, 1986.

APÊNDICE A – CÓDIGO-FONTE DO PROGRAMA DE TREINAMENTO

O Código 1 contém o programa completo `trabalho06_mlp.py`, usado para gerar todos os resultados deste relatório. Ele implementa a MLP 1–N–1, a retropropagação com momento, as inicializações uniforme e de Nguyen–Widrow e os experimentos da Seção 4. O treinamento base é executado com `python trabalho06_mlp.py`, e o conjunto completo de experimentos, com `python trabalho06_mlp.py --experimentos`. As únicas dependências são `numpy` e `matplotlib`.

Código 1 – Programa `trabalho06_mlp.py`.

```

1  #!/usr/bin/env python3
2  """
3  Trabalho 06 - Aproximação Funcional com MLP
4  EL056 - Redes Neurais Artificiais (PPGEELT/UFU) - Prof. Dr. Keiji Yamanaka
5  Aluno: Lucas A. Martins (12622EEL006)
6
7  MLP 1-N-1 (oculta tanh, saída linear) treinada com backpropagation
8  implementado do zero em NumPy: gradiente descendente com momento,
9  modos em lote (batch) e padrão a padrão (online), inicialização
10 uniforme ou Nguyen-Widrow.
11
12 Uso:
13     python trabalho06_mlp.py                # treinamento base + figura
14     python trabalho06_mlp.py --neurons 8 --lr 0.1 # outra configuração
15     python trabalho06_mlp.py --experimentos      # todos os experimentos do relatório
16 """
17 from __future__ import annotations
18
19 import argparse
20 import json
21 import os
22 from dataclasses import dataclass, field
23
24 import numpy as np
25 import matplotlib
26
27 matplotlib.use("Agg")
28 import matplotlib.pyplot as plt
29
30 # -----
31 # Dados do enunciado
32 # -----
33 X_DADOS = np.array([0.0, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1.0])
34 T_DADOS = np.array([-0.9602, -0.5770, -0.0729, 0.3771, 0.6405, 0.6600,
35                     0.4609, 0.1336, -0.2013, -0.4344, -0.5000])
36
37 # Configuração base (escolhida a partir dos experimentos)
38 BASE = dict(neurons=5, lr=0.05, momentum=0.9, epochs=20000, tol=1e-3,
39             seed=42, init="nguyen-widrow", mode="batch")
40
41
42 # -----
43 # Estruturas

```

```

44 # -----
45 @dataclass
46 class Params:
47     """Pesos da rede 1-N-1."""
48     W1: np.ndarray # (N, 1) entrada -> oculta
49     b1: np.ndarray # (N, 1) bias da oculta
50     W2: np.ndarray # (1, N) oculta -> saída
51     b2: np.ndarray # (1, 1) bias da saída
52
53     def copy(self) -> "Params":
54         return Params(self.W1.copy(), self.b1.copy(), self.W2.copy(), self.b2.copy())
55
56     def as_list(self) -> list[np.ndarray]:
57         return [self.W1, self.b1, self.W2, self.b2]
58
59
60 @dataclass
61 class Resultado:
62     params: Params
63     historico: list[float] = field(default_factory=list) # MSE por época
64     epocas: int = 0
65     convergiu: bool = False
66     divergiu: bool = False
67
68     @property
69     def mse_final(self) -> float:
70         return self.historico[-1] if self.historico else float("nan")
71
72
73 # -----
74 # Inicialização
75 # -----
76 def init_weights(n_hidden: int, metodo: str, rng: np.random.Generator) -> Params:
77     """Inicializa os pesos.
78
79     - 'uniforme': todos os pesos e biases em U(-0.5, 0.5).
80     - 'nguyen-widrow': pesos da oculta reescalados para norma  $\beta = 0.7 * N^{1/n}$ 
81       ( $n = n^\circ$  de entradas = 1) e biases em U(-beta, beta), espalhando as regiões
82       ativas das tanh pelo intervalo de entrada (Nguyen & Widrow, 1990).
83     """
84     n_in = 1
85     W1 = rng.uniform(-0.5, 0.5, (n_hidden, n_in))
86     b1 = rng.uniform(-0.5, 0.5, (n_hidden, 1))
87     if metodo == "nguyen-widrow":
88         beta = 0.7 * n_hidden ** (1.0 / n_in)
89         W1 = beta * W1 / np.linalg.norm(W1, axis=1, keepdims=True)
90         b1 = rng.uniform(-beta, beta, (n_hidden, 1))
91     elif metodo != "uniforme":
92         raise ValueError(f"método de inicialização desconhecido: {metodo}")
93     W2 = rng.uniform(-0.5, 0.5, (1, n_hidden))
94     b2 = rng.uniform(-0.5, 0.5, (1, 1))
95     return Params(W1, b1, W2, b2)
96
97
98 # -----
99 # Propagação direta e retropropagação
100 # -----
101 def forward(x: np.ndarray, p: Params) -> tuple[np.ndarray, dict[str, np.ndarray]]:
102     """Propagação direta. x: (P,) ou (1, P). Retorna y (1, P) e cache."""

```

```

1103     X = np.atleast_2d(x).reshape(1, -1)          # (1, P)
1104     net1 = p.W1 @ X + p.b1                        # (N, P)
1105     h = np.tanh(net1)                             # (N, P)  ativação da oculta
1106     y = p.W2 @ h + p.b2                          # (1, P)  saída linear
1107     return y, {"X": X, "h": h}
1108
1109
1110 def backward(t: np.ndarray, y: np.ndarray, cache: dict[str, np.ndarray],
1111             p: Params) -> list[np.ndarray]:
1112     """Retropropagação do erro E = 1/(2P) * sum (t - y)^2.
1113
1114     delta2 = (y - t) (saída linear: f'(net) = 1)
1115     delta1 = (W2.T @ delta2) * (1 - h^2) (derivada da tanh)
1116     """
1117     X, h = cache["X"], cache["h"]
1118     P = X.shape[1]
1119     T = np.atleast_2d(t).reshape(1, -1)
1120     delta2 = (y - T)                             # (1, P)
1121     delta1 = (p.W2.T @ delta2) * (1.0 - h ** 2)   # (N, P)
1122     dW2 = delta2 @ h.T / P
1123     db2 = delta2.sum(axis=1, keepdims=True) / P
1124     dW1 = delta1 @ X.T / P
1125     db1 = delta1.sum(axis=1, keepdims=True) / P
1126     return [dW1, db1, dW2, db2]
1127
1128
1129 def mse(x: np.ndarray, t: np.ndarray, p: Params) -> float:
1130     y, _ = forward(x, p)
1131     return float(np.mean((np.ravel(t) - np.ravel(y)) ** 2))
1132
1133
1134 def predict(x: np.ndarray, p: Params) -> np.ndarray:
1135     return np.ravel(forward(x, p)[0])
1136
1137
1138 # -----
1139 # Treinamento
1140 # -----
1141 def train(x: np.ndarray, t: np.ndarray, neurons: int = 5, lr: float = 0.05,
1142         momentum: float = 0.9, epochs: int = 20000, tol: float = 1e-3,
1143         seed: int = 42, init: str = "nguyen-widrow", mode: str = "batch",
1144         params0: Params | None = None) -> Resultado:
1145     """Gradiente descendente com momento:
1146     dw(k) = -lr * grad + momentum * dw(k-1); w <- w + dw(k)
1147     mode='batch': uma atualização por época com o gradiente médio.
1148     mode='online': uma atualização por padrão, em ordem embaralhada a cada época.
1149     Para quando MSE < tol (tol=0 desativa) ou ao atingir `epochs`.
1150     """
1151     rng = np.random.default_rng(seed)
1152     p = params0.copy() if params0 is not None else init_weights(neurons, init, rng)
1153     vel = [np.zeros_like(w) for w in p.as_list()]
1154     res = Resultado(params=p)
1155
1156     for ep in range(1, epochs + 1):
1157         if mode == "batch":
1158             lotes = [np.arange(len(x))]
1159         elif mode == "online":
1160             lotes = [[i] for i in rng.permutation(len(x))]
1161         else:

```

```

162         raise ValueError(f"modo desconhecido: {mode}")
163
164     for idx in lotes:
165         y, cache = forward(x[idx], p)
166         grads = backward(t[idx], y, cache, p)
167         for w, v, g in zip(p.as_list(), vel, grads):
168             v *= momentum
169             v -= lr * g
170             w += v
171
172     e = mse(x, t, p)
173     res.historico.append(e)
174     if not np.isfinite(e) or e > 1e6:
175         res.divergiu = True
176         break
177     if tol > 0 and e < tol:
178         res.convergiu = True
179         break
180     res.epocas = len(res.historico)
181     return res
182
183
184 # -----
185 # Gráficos
186 # -----
187 def _eixos_estilo_slide(ax: plt.Axes) -> None:
188     """Eixos cruzando na origem, limites [-1, 1] e título 'Dados', como no slide."""
189     ax.set_xlim(-1, 1)
190     ax.set_ylim(-1, 1)
191     ax.spines["left"].set_position("zero")
192     ax.spines["bottom"].set_position("zero")
193     ax.spines["right"].set_color("none")
194     ax.spines["top"].set_color("none")
195     ax.set_xticks(np.arange(-1, 1.01, 0.2))
196     ax.set_yticks(np.arange(-1, 1.01, 0.2))
197     ax.tick_params(labelsize=7)
198     ax.set_title("Dados", fontsize=10, fontweight="bold")
199
200
201 def plot(p: Params | None, caminho: str, x: np.ndarray = X_DADOS,
202         t: np.ndarray = T_DADOS, faixa: tuple[float, float] = (0.0, 1.0)) -> None:
203     """Pontos amostrados (losangos azuis) e curva aprendida (vermelho grosso)."""
204     fig, ax = plt.subplots(figsize=(5, 4.2))
205     _eixos_estilo_slide(ax)
206     if p is not None:
207         xs = np.linspace(*faixa, 400)
208         ax.plot(xs, predict(xs, p), color="red", lw=4, zorder=1)
209         ax.plot(x, t, "D", mfc="none", mec="blue", ms=5, zorder=2)
210     fig.tight_layout()
211     fig.savefig(caminho, dpi=200)
212     plt.close(fig)
213
214
215 # -----
216 # Experimentos do relatório
217 # -----
218 def fmt_params(p: Params) -> dict[str, list]:
219     return {k: np.round(v, 4).tolist() for k, v in
220             zip(["W1", "b1", "W2", "b2"], p.as_list())}

```

```

221
222
223 def experimentos(pasta_fig: str = "figuras", saida_json: str = "resultados.json") -> dict:
224     os.makedirs(pasta_fig, exist_ok=True)
225     R: dict = {"base": BASE}
226     x, t = X_DADOS, T_DADOS
227
228     # 0) Dados brutos (figura esquerda do slide)
229     plot(None, f"{pasta_fig}/dados.png")
230
231     # 1) Treinamento base
232     print("[1] Treinamento base")
233     base = train(x, t, **BASE)
234     plot(base.params, f"{pasta_fig}/aproximacao_base.png")
235     R["treino_base"] = {"epocas": base.epocas, "mse": base.mse_final,
236                       "convergiu": base.convergiu, "pesos": fmt_params(base.params),
237                       "saidas": np.round(predict(x, base.params), 4).tolist()}
238     print(f"    épocas={base.epocas} MSE={base.mse_final:.2e}")
239
240     # Treino "até o fim" (sem tolerância) para ver até onde o erro cai
241     longo = train(x, t, **{**BASE, "tol": 0.0, "epochs": 50000})
242     R["treino_longo"] = {"epocas": longo.epocas, "mse": longo.mse_final}
243
244     # 2) Curva de aprendizado (base e longo)
245     fig, ax = plt.subplots(figsize=(6, 3.4))
246     ax.semilogy(longo.historico, color="gray", lw=1, label="sem parada antecipada")
247     ax.semilogy(base.historico, color="red", lw=2, label=f"base (para em MSE < {BASE['tol']:g})")
248     ax.axhline(BASE["tol"], ls="--", color="k", lw=0.8)
249     ax.set_xscale("log"); ax.set_xlabel("Época (escala log)"); ax.set_ylabel("MSE"); ax.grid(True,
250     which="both", alpha=0.3)
251     ax.legend(fontsize=8); fig.tight_layout()
252     fig.savefig(f"{pasta_fig}/curva_aprendizado.png", dpi=200); plt.close(fig)
253
254     # 3) Número de neurônios ocultos (treino sem tolerância, 50000 épocas)
255     print("[3] Variação de neurônios")
256     Ns = [1, 2, 3, 5, 10, 20]
257     xs = np.linspace(0, 1, 400)
258     R["neuronios"] = []
259     fig, axs = plt.subplots(2, 3, figsize=(9, 5.4), sharex=True, sharey=True)
260     for ax, N in zip(axs.ravel(), Ns):
261         r = train(x, t, **{**BASE, "neurons": N, "tol": 0.0, "epochs": 50000})
262         R["neuronios"].append({"N": N, "mse": r.mse_final})
263         ax.plot(xs, predict(xs, r.params), "r", lw=2)
264         ax.plot(x, t, "D", mfc="none", mec="blue", ms=4)
265         ax.set_title(f"N = {N} MSE = {r.mse_final:.1e}", fontsize=9)
266         ax.set_ylim(-1.1, 1.0); ax.grid(alpha=0.3)
267         print(f"    N={N:2d} MSE={r.mse_final:.2e}")
268     fig.tight_layout(); fig.savefig(f"{pasta_fig}/neuronios.png", dpi=200); plt.close(fig)
269
270     # 4) Taxa de aprendizado x momento (épocas até MSE < tol; mediana de 10 sementes)
271     print("[4] Taxa de aprendizado x momento")
272     lrs, moms, seeds = [0.01, 0.05, 0.1, 0.3], [0.0, 0.5, 0.9], range(10)
273     R["lr_momento"] = []
274     for lr in lrs:
275         for m in moms:
276             eps, ok, div = [], 0, 0
277             for s in seeds:
278                 r = train(x, t, **{**BASE, "lr": lr, "momentum": m, "seed": s, "epochs": 50000})
279                 div += r.divergiu

```

```

279         if r.convergiu:
280             ok += 1; eps.append(r.epocas)
281             R["lr_momento"].append({"lr": lr, "momento": m, "convergiram": ok,
282                                     "divergiram": div, "total": len(seeds),
283                                     "epocas_mediana": float(np.median(eps)) if eps else None})
284             print(f"    lr={lr:<5} mom={m:<4} conv={ok}/10 div={div} "
285                   f"mediana={np.median(eps) if eps else float('nan'):.0f}")
286
287     # 4b) Inicialização e modo de treinamento (10 sementes)
288     print("[4b] Inicialização e modo")
289     R["init_modos"] = []
290     for init in ["uniforme", "nguyen-widrow"]:
291         for mode in ["batch", "online"]:
292             eps, ok = [], 0
293             for s in range(10):
294                 r = train(x, t, **{**BASE, "init": init, "mode": mode, "seed": s, "epochs": 50000})
295                 if r.convergiu:
296                     ok += 1; eps.append(r.epocas)
297                 R["init_modos"].append({"init": init, "modo": mode, "convergiram": ok, "total": 10,
298                                         "epocas_mediana": float(np.median(eps)) if eps else None})
299             print(f"    {init:14s} {mode:7s} conv={ok}/10 mediana={np.median(eps) if eps else float('nan'):.0f}")
300
301     # 5) Generalização: leave-one-out (LOO) para cada N
302     print("[5] Leave-one-out")
303     R["loo"] = []
304     for N in Ns:
305         erros = []
306         for i in range(len(x)):
307             m = np.ones(len(x), bool); m[i] = False
308             r = train(x[m], t[m], **{**BASE, "neurons": N, "tol": 0.0, "epochs": 20000})
309             erros.append((predict(x[i:i + 1], r.params)[0] - t[i]) ** 2)
310         R["loo"].append({"N": N, "mse_loo": float(np.mean(erros)),
311                         "max_erro": float(np.sqrt(np.max(erros)))})
312         print(f"    N={N:2d} MSE_LOO={np.mean(erros):.2e}")
313     fig, ax = plt.subplots(figsize=(5.5, 3.2))
314     mse_tr = [max(d["mse"], 1e-7) for d in R["neuronios"]] # piso só para visualização
315     ax.semilogy(Ns, mse_tr, "o-", label="MSE de treino")
316     ax.semilogy(Ns, [d["mse_loo"] for d in R["loo"]], "s-", color="red", label="MSE leave-one-out")
317     ax.set_xlabel("Neurônios na camada oculta (N)"); ax.set_ylabel("MSE")
318     ax.set_ylim(5e-8, 1); ax.set_xticks(Ns); ax.grid(True, which="both", alpha=0.3)
319     ax.annotate("treino  $\approx$  0\n(interpolação)", (20, 1e-7), (14, 1e-5), fontsize=7,
320               arrowprops=dict(arrowstyle=">", lw=0.8))
321     ax.legend(fontsize=8)
322     fig.tight_layout(); fig.savefig(f"{pasta_fig}/loo.png", dpi=200); plt.close(fig)
323
324     # 6) Extrapolação em [-1, 1]
325     print("[6] Extrapolação")
326     fig, ax = plt.subplots(figsize=(5, 4.2))
327     _eixos_estilo_slide(ax); ax.set_ylim(-1.5, 1.0)
328     ax.set_yticks(np.arange(-1.5, 1.01, 0.5))
329     xe = np.linspace(-1, 1, 400)
330     for N, cor in [(3, "tab:green"), (5, "red"), (20, "tab:purple")]:
331         r = train(x, t, **{**BASE, "neurons": N, "tol": 0.0, "epochs": 50000})
332         ax.plot(xe, predict(xe, r.params), color=cor, lw=2, label=f"N = {N}")
333     ax.xspan(0, 1, color="yellow", alpha=0.15, lw=0)
334     ax.plot(x, t, "D", mfc="none", mec="blue", ms=5)
335     ax.legend(fontsize=8, loc="lower right"); fig.tight_layout()
336     fig.savefig(f"{pasta_fig}/extrapolacao.png", dpi=200); plt.close(fig)

```

```

337
338     with open(saida_json, "w", encoding="utf-8") as f:
339         json.dump(R, f, indent=2, ensure_ascii=False)
340     print(f"Resultados salvos em {saida_json}; figuras em {pasta_fig}/")
341     return R
342
343
344     # -----
345     # CLI
346     # -----
347     def main() -> None:
348         ap = argparse.ArgumentParser(description="Trabalho 06 - Aproximação funcional com MLP")
349         ap.add_argument("--neurons", type=int, default=BASE["neurons"])
350         ap.add_argument("--lr", type=float, default=BASE["lr"])
351         ap.add_argument("--momentum", type=float, default=BASE["momentum"])
352         ap.add_argument("--epochs", type=int, default=BASE["epochs"])
353         ap.add_argument("--tol", type=float, default=BASE["tol"])
354         ap.add_argument("--seed", type=int, default=BASE["seed"])
355         ap.add_argument("--init", choices=["uniforme", "nguyen-widrow"], default=BASE["init"])
356         ap.add_argument("--mode", choices=["batch", "online"], default=BASE["mode"])
357         ap.add_argument("--saida", default="aproximacao.png", help="arquivo da figura")
358         ap.add_argument("--experimentos", action="store_true",
359                         help="roda todos os experimentos do relatório")
360         a = ap.parse_args()
361
362         if a.experimentos:
363             experimentos()
364             return
365
366         r = train(X_DADOS, T_DADOS, a.neurons, a.lr, a.momentum, a.epochs,
367                 a.tol, a.seed, a.init, a.mode)
368         status = "convergiu" if r.convergiu else ("DIVERGIU" if r.divergiu else "máx. de épocas")
369         print(f"MLP 1-{a.neurons}-1 | {status} em {r.epocas} épocas | MSE = {r.mse_final:.4e}")
370         print(" x      alvo      saída")
371         for xi, ti, yi in zip(X_DADOS, T_DADOS, predict(X_DADOS, r.params)):
372             print(f"{xi:.1f}  {ti:+.4f}  {yi:+.4f}")
373         plot(r.params, a.saida)
374         print(f"Figura salva em {a.saida}")
375
376
377     if __name__ == "__main__":
378         main()

```